Multi Agent Systems with LangGraph and CrewAI

Demo 2

Demo: Design a Stock Analysis Agent with CrewAI

Overview

This demonstration highlights the design and execution of a multi-agent system utilizing CrewAI to conduct stock analysis for Apple Inc. (AAPL). The system utilizes CrewAI's structure to develop tailored agents that work together to collect news, assess stock price patterns, and produce reports suitable for investors. The initiative combines external APIs (such as Yahoo Finance, DuckDuckGo) with a large language model (Gemini) to provide practical insights. This practical activity corresponds with the module "Multi Agent Systems with LangGraph and CrewAI," concentrating on creating cooperative agents for actual applications.

Scenario

Business: FinTech Analytics Corporation.

Position: AI Systems Engineer

Context: FinTech Analytics Inc. is a startup in the financial technology sector focused on offering automated stock analysis tools for retail investors. The firm has assigned you the responsibility of creating a multi-agent system to evaluate stocks, beginning with Apple Inc. (AAPL). The system must retrieve current news, examine price movements, and generate a brief report for investors. The answer needs to be scalable, modular, and compatible with external data sources to provide real-time insights.

Problem Statement

Create a multi-agent system that independently collects current news regarding a stock (AAPL), examines its price movements from the past month, and produces a formal investor report highlighting the results.

Obstacles:

- Data Integration: Consistently retrieving and handling real-time information from outside APIs (Yahoo Finance, DuckDuckGo).
- Agent Cooperation: Making certain that agents collaborate effectively to exchange information and generate a consistent result.
- Scalability: Creating a modular framework capable of managing extra stocks or tasks.
- API Key Administration: Safely managing API keys for the Gemini LLM and additional services.
- Error Management: Addressing possible issues in API requests or data handling.
- Output Quality: Generating a succinct report suitable for investors that harmonizes technical specifics with clarity.

Approach to the Solution

The approach employs CrewAI to create a multi-agent system consisting of two agents:

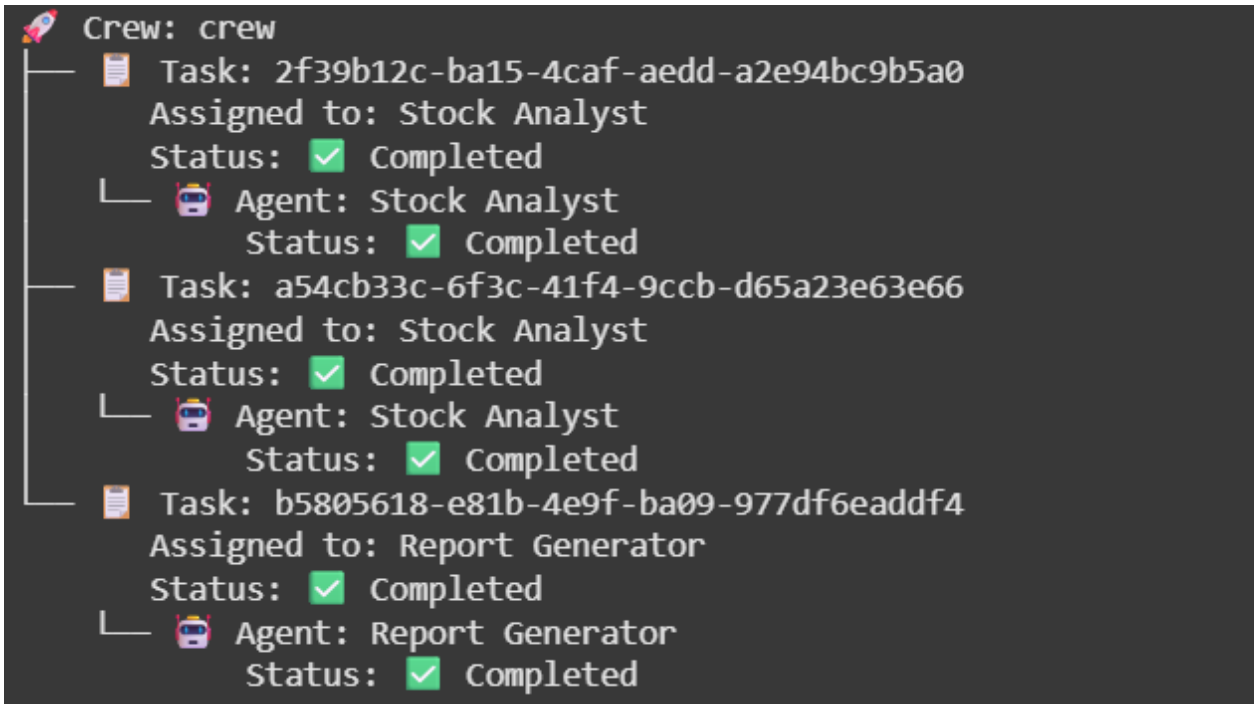
- Stock Analyst Agent: Tasked with retrieving news (through DuckDuckGo) and examining price trends (using Yahoo Finance).
- Report Generation Agent: Creates a professional investor report grounded in the Stock Analyst's results.

Steps:

1. Establish the CrewAI setup and its required dependencies.
2. Create personalized tools for news searching and stock price fetching.
3. Develop characters with distinct roles, objectives, and histories.
4. Allocate responsibilities to agents for gathering data, conducting analysis, and producing reports.
5. Gather a team to coordinate agent teamwork.
6. Carry out the workflow and produce the final report.

Instruments and Application Programming Interfaces:

- CrewAI: Framework for orchestrating multiple agents.
- Yahoo Finance (yfinance): For data on stock prices.
- DuckDuckGo Search: For up-to-date news.
- Gemini LLM: Designed for processing natural language and creating reports.



Stepwise Solution

Project Structure:

```
stock_analysis_project/
├── .env                    # Environment variables (API keys)
├── stock_analysis.ipynb    # Jupyter Notebook with the implementation
├── README.md               # Project documentation
└── requirements.txt        # Project dependencies
```

NOTE: This Demonstration has been performed on Google Colab Notebook.

Step 1: Installation and Setup

Prerequisites

- Python 3.8+
- A Google Gemini API key (obtain from Google Cloud or Gemini platform)
- Install required libraries: crewai, yfinance, duckduckgo_search, python-dotenv, google-generativeai, crewai-tools

```
pip install -q crewai google-generativeai duckduckgo_search yfinance pandas beautifulsoup4 python-dotenv
pip install -q crewai-tools
```

Or, requirements.txt file

```
crewai
google-generativeai
duckduckgo_search
yfinance
pandas
beautifulsoup4
python-dotenv
crewai-tools
```

API Key Setup

1. Create a .env file in the project root:

GEMINI_API_KEY=your_gemini_api_key_here

2. Alternatively, input the key securely during runtime using getpass.

Step 2: Import Libraries

```
import os
from dotenv import load_dotenv
from crewai import Agent, Task, Crew, LLM
import yfinance as yf
from duckduckgo_search import DDGS
from crewai.tools import BaseTool
```

- crewai: Provides Agent, Task, Crew, and LLM classes for multi-agent systems.
- yfinance: Fetches stock price data.
- duckduckgo_search: Performs web searches for news.
- BaseTool: Base class for custom CrewAI tools.

Step 3: Load Gemini API Key

```
from getpass import getpass
os.environ["GEMINI_API_KEY"] = getpass("Enter your Gemini API Key: ")
```

- Securely load the Gemini API key from the environment or prompt the user.
- Uses getpass to securely input the API key during runtime, avoiding hardcoding sensitive information.

Step 4: Setup Gemini LLM

```
llm = LLM(
    model="gemini/gemini-2.0-flash",
    api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.7
)
```

- Initializes the Gemini LLM with the gemini-2.0-flash model.
- Sets temperature=0.7 for balanced creativity and coherence.
- Links the API key from the environment.

Step 5: Define Custom Tools

```
class StockSearchTool(BaseTool):
    name: str = "StockNewsSearcher"
    description: str = "Search for the latest news and updates about a stock using DuckDuckGo"

    def _run(self, query: str) -> str:
        with DDGS() as ddgs:
            results = ddgs.text(query)
            return "\n".join([r['body'] for r in results[:3]])

class YahooFinanceTool(BaseTool):
    name: str = "YahooFinanceFetcher"
    description: str = "Get the latest 1-month stock price history for a given ticker using yFinance"

    def _run(self, ticker: str) -> str:
        stock = yf.Ticker(ticker)
        hist = stock.history(period="1mo")
        return hist.tail().to_string()
```

- StockSearchTool: Uses duckduckgo_search to fetch the top 3 news snippets for a given query (e.g., "AAPL stock news").
- YahooFinanceTool: Retrieves the last 5 days of 1-month stock price history for a ticker using yfinance.
- Both tools inherit from BaseTool and implement the _run method.

Step 6: Define Agents

```
stock_analyst = Agent(
    role='Stock Analyst',
    goal='Analyze recent stock data and news',
    backstory='Expert in financial trends, macro indicators, and company performance',
    verbose=True,
    allow_delegation=False,
    llm=llm
)

report_writer = Agent(
    role='Report Generator',
    goal='Write investor-friendly summaries of stock analysis',
    backstory='Professional writer with expertise in finance reporting',
    verbose=True,
    allow_delegation=False,
    llm=llm
)
```

- Stock Analyst: Focuses on data collection and analysis, using the custom tools.
- Report Writer: Synthesizes the analyst's findings into a polished report.
- allow_delegation=False ensures each agent handles its tasks independently.
- Both agents use the Gemini LLM for processing.

Step 7: Create Tasks

Define tasks for news search, price analysis, and report generation.

```
search_tool = StockSearchTool()
finance_tool = YahooFinanceTool()

search_task = Task(
    description="Search latest news and updates about the stock 'AAPL' using DuckDuckGo.",
    expected_output="Summarized news highlights for Apple stock.",
    agent=stock_analyst,
    tools=[search_tool]
)

analysis_task = Task(
    description="Analyze Apple stock price trends using yFinance.",
    expected_output="Key trends and technical highlights for the past month.",
    agent=stock_analyst,
    tools=[finance_tool]
)

report_task = Task(
    description="Write a clean investor report using previous analysis and news insights.",
    expected_output="Concise report with market summary and investment outlook.",
    agent=report_writer
)
```

- Search Task: Assigns the Stock Analyst to fetch AAPL news using StockSearchTool.
- Analysis Task: Directs the Stock Analyst to analyze AAPL price trends using `Yahoo surprisingly, the code cuts off here. I'll continue from where it left off and complete the documentation.

Step 8: Assemble the Crew

Orchestrate the agents and tasks using a Crew.

```
crew = Crew(
    agents=[stock_analyst, report_writer],
    tasks=[search_task, analysis_task, report_task],
    verbose=True
)
```

- The Crew class coordinates the execution of tasks by the assigned agents.
- verbose=True enables detailed logging of the execution process.

- Verbose=False reduces rich console output and avoid RecursionError

Step 9: Run the Agent Crew

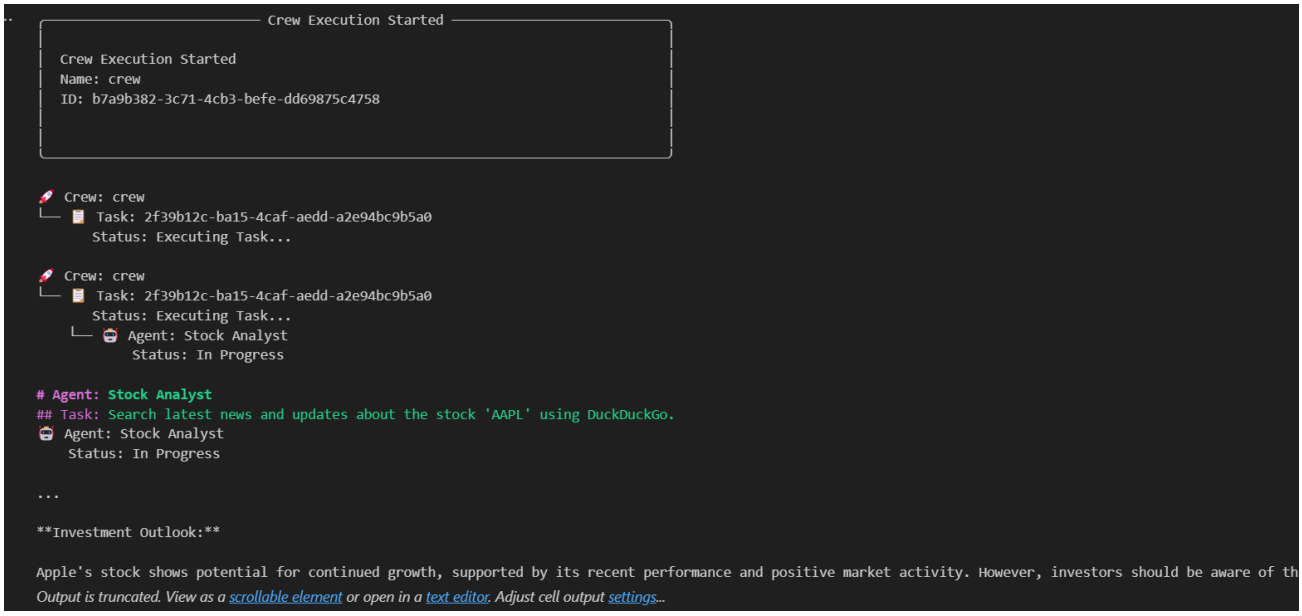
Execute the workflow and display the final report.

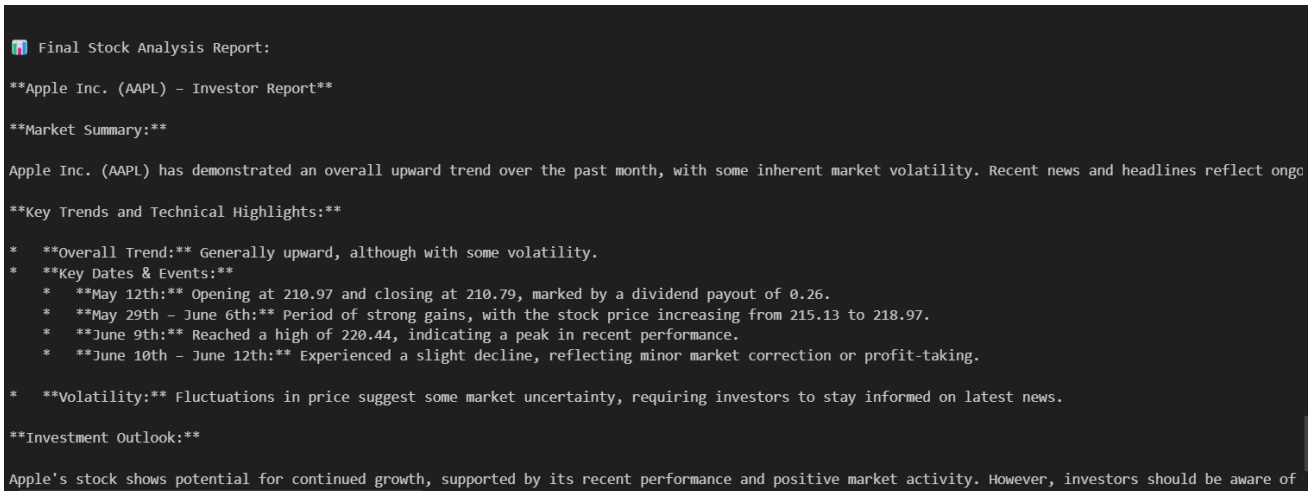
```
result = crew.kickoff()

print("\n📊 Final Stock Analysis Report:\n")
print(result)
```

- crew.kickoff() triggers the sequential execution of tasks.
- The final output (the investor report) is printed for review.

Output

The system produces a report like this:





Map with Topics

This showcase corresponds with the subsequent subjects in the "Multi Agent Systems with LangGraph and CrewAI" module:

- **Multi-Agent Systems**: The project utilizes a multi-agent system featuring distinct roles (Stock Analyst, Report Writer) working together to reach a shared objective.
- **Multi-Agent Workflows**: The sequential workflow (news search → price evaluation → report creation) illustrates task coordination in a multi-agent framework.

- **Collaborative Multi Agents**: Agents convey insights indirectly via task results, while the Report Writer combines the Stock Analyst's conclusions.
- **Multi-Agent Frameworks**: The architecture employs a modular structure with tailored utilities and defined agent roles, facilitating scalability.
- **CrewAI**:
    - Presents CrewAI as a structure for developing multi-agent systems.
    - Employs fundamental CrewAI elements: Agents, Tasks, Tools, and Crew.
    - Involves installing CrewAI and its dependencies, as well as configuring the API key.
    - Shows the development of agents featuring roles, objectives, narratives, and LLM integration.
- **Multi-Agent Workflow with LangGraph** (Not directly utilized): Although LangGraph isn't employed in this instance, the workflow principles correspond with graph-oriented task orchestration ideas. Demo 1 focuses on Multiagent system with LangGraph.

This practical demonstration showcases the capability of CrewAI in creating a multi-agent system for stock evaluation. By incorporating external APIs (Yahoo Finance, DuckDuckGo) and the Gemini LLM, the platform provides valuable insights for investors. The modular structure allows for scalability, and the distinct allocation of agent roles improves maintainability.